Semantic Contracts for HE ↔ PeTTa Translation

Shared Core, Compatibility Profiles, and Production Boundaries

Zar Goertzel

May 2026

Abstract

This document states the semantic contract for translating between Hyperon Experimental (HE) MeTTa and PeTTa MeTTa. The central point is that both systems use MeTTa syntax, but they do not always give the same operational meaning to every surface form. A faithful translator must therefore do more than parse and reprint S-expressions: it must preserve observable behavior on a shared core, lower source conveniences to explicit target forms where possible, and classify helper or profile-specific surfaces explicitly when portability requires more than syntax.

We describe the shared and non-shared language surfaces, the main translation rules in both directions, the status of quote/eval/reduce, `test`, `hyperpose`, type queries, state, imports, and space operations, and the validation obligations for production use. The companion engineering specification lives with the translator implementation at `hyperon/translators/specs/he-petta-translation-semantics.md`.

Contents

1 Problem

HE and PeTTa are both MeTTa runtimes, but MeTTa syntax alone does not fix all runtime behavior. A file that ends in `.metta` may be:

1. a source program intended for default PeTTa;
2. an HE program intended for a reference HE runtime;
3. a generated HE-facing artifact intended for `petta -he`;
4. an artifact that supplies helper definitions to preserve source behavior;
5. an extension program that relies on host facilities outside core HE.

The translator's job is to keep these categories explicit. Classifying every generated artifact as pure HE would be incorrect. Classifying every difference as an untranslatable failure would also be incorrect, because many differences have principled lowerings.

Positive example. The PeTTa source form

```
!(test (length (collapse (demo-peano 300))) 301)
```

can be translated to an explicit HE-facing computation:

```
!(test (let $__tr_tuple_1 (collapse (demo-peano 300))
        (size-atom $__tr_tuple_1))
      301)
```

This is ordinary portable lowering.

Negative example. The PeTTa source form

```
!(test (quote (+ 1 2)) (+ 1 2))
```

depends on PeTTa's visible quote behavior. A target runtime whose `quote` returns a visible wrapper cannot be claimed to agree merely because both sides parse the same text.

2 Semantic Lanes

The translator uses the following semantic lanes.

Lane	Meaning	Example
Shared core	Behavior is expected to agree on the target engines under consideration.	`if`, `case`, many `let`, `match`, and arithmetic examples.
Portable lowering	Source surface is rewritten to target forms without PeTTa-only assumptions.	`length (collapse e)` to `let` plus `size-atom`.
Translator helper	Generated artifact needs a small helper definition to preserve source behavior.	`test`, `assertEqualToEval`, PeTTa-profile `quoted-syntax`.
HE-extended compatibility	Uses an HE-family surface present in CeTTa `he-extended` and PeTTa `-he`, but not upstream HE core.	`select 1` in an explicit extended lane.
PeTTa HE compatibility	Intended for `./run.sh -he`; may rely on profile behavior outside HE core.	Callable-return application and selected quote/eval/reduce compatibility.
PeTTa extension	Useful runtime surface outside the strict portability claim.	`cut`, native `msort`, Python FFI, host resources, selected path/indexing support.

This table is part of the correctness contract. A helper-class artifact may be correct for its intended target while remaining unsuitable as a backend-agnostic HE conformance example.

3 HE Surface

The HE target surface used by the translator is centered on explicit evaluation and binding constructs. Important target forms include:

Construct	Role
`chain`	Sequenced evaluation with result binding.
`collapse`, `collapse-bind`	Collection of nondeterministic results.
`superpose`, `superpose-bind`	Re-entry of alternatives.
`eval`, `evalc`, `unquote`	Explicit evaluation of syntax/data.
`case`, `switch`	Pattern-directed branching.
`size-atom`	Explicit tuple size/count surface.
`function`, `return`	Minimal function-return surface.
`metta`	Interpreter call with expected type and space.

The translator should not turn implementation extensions into HE core requirements. If a target runtime provides an extra surface, that surface must be named as a profile or extension.

4 PeTTa Surface

PeTTa source programs use the same S-expression syntax plus Prolog-backed runtime conveniences. Some forms are shared. Others require explicit translation.

PeTTa surface	Translation status
`let`, `let*`, `if`, `case`, `match`, `collapse`, `superpose`	Usually shared or nearly identity.
`progn`, `prog1`	Lowered to sequenced `chain`/`let` forms.
`foldall`, PeTTa-style `foldl-atom`	Lowered to `collapse` plus explicit accumulator/item variables.
`length (collapse e)`	Lowered to `let tuple (collapse e') (size-atom tuple)`.
`test`	Lowered to self-contained observable test behavior, because its printed report is observable behavior.
`quote`, `eval`, `reduce`	Compatibility lane; see Section **??**.
`hyperpose`	Lowered to `superpose` by default, preserved only with explicit target intent.
Python, Git, and host FFI	Passed through or extension-classified; not pure HE.

5 HE to PeTTa

The HE-to-PeTTa direction adapts explicit HE sequencing and binding to PeTTa source forms:

HE input	PeTTa output	Intent
`(chain e $x b)`	`(let $x e b)`	Preserve sequencing.
`(collapse-bind e)`	`(collapse e')`	Preserve collection.
`(superpose-bind xs)`	`(superpose xs')`	Preserve alternatives.
`(nop e)`	`(let $fresh e' ())`	Keep effect, discard result.
`(switch x bs)`	`(case x' bs')`	Translate branches.
`(function (return x))`	`x'`	Remove wrapper where safe.

Fresh names use a reserved generated namespace such as `$__tr_...`. Correctness requires capture avoidance: generated names must not accidentally bind or shadow source variables.

6 PeTTa to HE

The PeTTa-to-HE direction lowers source conveniences to explicit HE-facing forms:

PeTTa input	HE-facing output	Intent
`progn`	Nested sequencing	Preserve effects and final value.
`prog1`	Result capture plus sequencing	Preserve first result.
`foldall`	`collapse` plus `foldl-atom`	Make fold explicit.
`foldl-atom`	Explicit accumulator/item variables	Avoid implicit agg shape.
`length (collapse e)`	`let` plus `size-atom`	Portable lowering.
`test`	File-local `test` helper	Preserve observable test behavior without assuming a target-native PeTTa `test`.
`once`	`collapse` plus `case`/`decons-atom` by default	Preserve committed choice without relying on `select`; explicit extended and PeTTa profile lanes may use their native surfaces.
`unique-atom (collapse e)`	`collapse (unique e')`	Use target uniqueness.
`hyperpose`	`superpose`, unless preserved	Drop scheduling, keep values.

7 Quote, Eval, Reduce

Quote/eval/reduce are a common compatibility boundary. In default PeTTa,

```
!(quote (+ 1 2))
```

exposes the syntax value:

```
(+ 1 2)
```

whereas some HE implementations expose a visible quote wrapper:

```
(quote (+ 1 2))
```

The pure PeTTa-to-HE translator lane uses native HE quote and explicit unquote evaluation:

```
(quote e) -> (quote e')
(call e) -> (unquote (quote e'))
(eval e) -> (unquote (quote e'))
(reduce e) -> (unquote (quote e'))
```

When the source expression is a variable expected to contain quoted code, the pure lane lowers to `(unquote $code)`.

The PeTTa `-he` profile lane may instead preserve PeTTa-visible raw quoted syntax by inserting a helper:

```
(= (quoted-syntax (quote $expr)) $expr)
```

and lowering `(quote e)` to `(quoted-syntax (quote e'))`. That helper is a profile compatibility surface, not a pure-HE core claim.

8 Tests And Assertions

PeTTa's `test` form prints an actual/expected report before returning the assertion result. That output is observable behavior. Therefore the translator preserves source `test` behavior by default in the generated artifact itself by supplying a file-local `test` helper when the source uses PeTTa's builtin test.

```
!(test (+ 1 2) 3)
```

lowers to a self-contained helper-backed assertion:

```
(: test (-> Atom Atom Bool))
(= (test $actual $expected) ...)
!(test (+ 1 2) 3)
```

Quiet assertion helpers are appropriate for explicit profile tests. Examples include `assertEqualToEval`, `assertEqualToResult`, and `assertAlphaEqualToResult`. They are not a faithful replacement for a source `test` whose printed report matters.

Likewise, generated portable artifacts must not rely on another engine providing PeTTa's private `test` builtin. If a generated file contains `test`, it must also supply the compatible helper definition or import a declared compatibility library.

9 Committed Choice And Cut

PeTTa's `once` is committed choice over a nondeterministic expression. The default pure translator lane lowers it through HE core building blocks: `collapse`, `case`, and `decons-atom`. An explicit extended lane may lower `(once expr)` to `(select 1 expr')`; that is an HE-extended compatibility claim, not an upstream-HE-core claim.

The explicit PeTTa profile lane, selected with `translate.sh petta2he -petta-he`, preserves `once`. That lane is for artifacts intended to run under PeTTa `-he`, including performance-sensitive examples that rely on the profile's direct committed choice implementation.

PeTTa `cut` is search-control rather than a first-result operator on one expression. A target that preserves `cut` is a PeTTa-profile target. The default pure translator rejects general `cut` rather than lowering it to an ordinary value, `once`, or a whole-expression first result. A future portable translation would need a specified ordered-commit semantics, not a local value rewrite.

PeTTa-native `msort` is likewise outside HE core unless the source program defines `msort` itself. It depends on the source runtime's order over arbitrary atoms. The pure translator rejects unprovided `msort`; the PeTTa profile lane may preserve it for runtimes that implement the native ordering.

10 Hyperpose

`hyperpose` is PeTTa's parallel nondeterministic surface. The default portable translation lowers it to `superpose`, preserving the nondeterministic values but not claiming to preserve parallel scheduling.

```
(hyperpose xs) -> (superpose xs')
```

When the target runtime intentionally supports `hyperpose`, users may ask the translator to preserve it:

```
./translate.sh petta2he --preserve-hyperpose input.metta output.metta
```

Such a file is no longer the default pure-portable lowering. It is a runtime-profile artifact.

11 Type Queries

The translator depends on unknown type behavior being stable. The intended behavior is:

```
!(get-type $x) ; %Undefined%
!(get-metatype $x) ; Variable
```

Returning a fresh type variable for `(get-type $x)` is unsound for translation: that variable can escape into stored type facts and later unify with concrete types, producing branch-order-dependent routes. Unknown types should remain unknown; metatypes distinguish variables.

12 Imports, Spaces, And State

Translation over imports is supported in recursive and bundle modes. Space and state operations are part of the semantic boundary because they can expose mutation order, duplicate atoms, backend indexing, and host resources.

PeTTa named state and HE native state share spellings such as `new-state`, `get-state`, and `change-state!`, but they are not the same portable contract. A PeTTa form such as:

```
!(bind! counter (new-state 0))
```

uses `counter` as a mutable cell name. HE native state allocates an explicit state handle. The PeTTa-to-HE direction therefore lowers named state through translator helpers instead of treating it as identity syntax.

The rule is simple: preserve or lower what has a specified target meaning, and classify the rest. Python handles, Git imports, MORK/PathMap handles, runtime statistics, and other host resources are not pure HE merely because the syntax is readable by an HE parser.

13 Where the Engines Actually Differ (Measured)

The preceding sections state the contract abstractly. This section records the concrete semantic divergences observed empirically between upstream HE and PeTTa `-he` — the points where MeTTa syntax coincides but operational meaning does not.

Method. Each HE surface is run on upstream Rust HE (the sole oracle of HE meaning) and on PeTTa `-he` (the subject under test); result lines are compared after safe normalization only. Over the 34 canonical HE example programs (the `a`–`g` families), PeTTa `-he` reproduces the upstream result on **26 of 34**; the 8 divergences are not scattered but cluster in the families below, and per-surface probes corroborate and add finer cases (such as the `if`-arity row). The machine-readable record is the surface registry `petta-he-profile/tests/he_spec_surfaces.tsv` and the divergence registry `petta-he-profile/src/he/DIVERGENCES.md` (measured 2026-06).

Family	Difference (HE vs PeTTa)	Status
Source-head no-match	HE keeps an unmatched source-head application inert (arguments evaluated); PeTTa returns the empty result. This is the foundational difference and the reason the `call-or-inert` helper exists.	bridged by helper
`quote`	PeTTa's `quote` reduces to its argument (`(quote X)` → `X`); HE preserves the quoted form, changing match-template and eval staging.	bridged by helper
Arity strictness	HE errors `IncorrectNumberOfArguments` on a two-argument `if`; PeTTa `-he` evaluates it as if-then (`(if True 42)` → `42`).	DIV-011; leniency
Typed-error surface	HE emits structured `BadArgType`/`BadType`; PeTTa is more permissive (inferred type, `%Undefined%`, or empty), and grounded arithmetic on non-numbers must be guarded to avoid a host abort.	`b5`; leniency
Recursive reduction	HE reduces post-match templates and recursive equality rules to a fixpoint; PeTTa one-step post-match evaluation can leave a compound recursive template unreduced.	`b1`, `d4`; genuine gap
Higher order	HE hand-rolls currying (no native partial application); PeTTa applies partials natively, so the encodings differ.	`d2`
Imports and spaces	Module scoping, transitive-import materialization, and printed variable naming (gensym vs source name) differ.	`f1`; engine boundary
Documentation	`get-doc` arity and behavior differ by build.	`g1`; minor

Consequence. PeTTa `-he` is faithful to upstream HE on the broad shared core (arithmetic, `eval`, `chain`, `cons-atom`/`decons-atom`, defining equations, backchaining, nondeterminism, spaces, simple type queries, state), but falls back to PeTTa semantics on the families above — precisely the surfaces where the two dialects genuinely disagree. PeTTa `-he` is therefore a useful compatibility profile, *not* a faithful HE reference: it must not be used as an HE oracle or as evidence of HE conformance. For any question about HE meaning, upstream Rust HE is the oracle and `-he` is a

subject under test. Making `-he` faithful on these families would mean implementing HE's distinctive reduction and typing behavior inside PeTTa, which is outside the compatibility-profile scope.

14 Evidence And Gates

There are several distinct evidence layers:

- `translate.sh -test`: unit and real-file translator behavior.
- `run_he_profile_suite.sh`: PeTTa `-he` runtime profile behavior.
- `run_translator_survey.sh`: corpus correctness and performance budget.
- `run_pure_he_engine_bench.sh`: cross-engine checks for freshly generated pure outputs.
- `check_generated_he_portability.sh`: cross-engine classification for PeTTa-profile artifacts.
- Lean translator files: proof-oriented common-fragment correspondence.

These gates answer different questions. A green PeTTa `-he` survey is not automatically a cross-engine proof. A helper-surface classification is not a runtime failure. A proof over the common fragment does not license arbitrary host extensions.

15 Production Criteria

A translation rule is production-ready only when:

1. its source and target semantic lanes are named;
2. positive and negative examples are recorded;
3. fresh variables are capture-avoiding;
4. generated helper names avoid source-symbol collisions;
5. observable output is preserved or explicitly classified;
6. helper surfaces are named as helpers, not HE core;
7. translator tests pass;
8. representative generated artifacts run under their intended target;
9. cross-engine differences are classified when portability is claimed.

Performance work must preserve these correctness obligations. A faster translation that changes source behavior is a regression.

16 Conclusion

The HE $\leftrightarrow$ PeTTa translator should be understood as a semantic bridge between related MeTTa profiles, not as a string rewriter between identical languages. Its most important discipline is classification: shared core behavior, portable lowering, helper surfaces, compatibility profiles, and extensions must stay distinct. With that discipline, the translator can serve both engineering use and formalization work without making false portability claims.